

Team Coupon Hunting

- ให้นักเรียนจับกลุ่ม กลุ่มละ 5-6 คน และตกลงแบ่งหน้าที่เพื่อจำลองระบบสถาปัตยกรรม Client-Server ดังนี้
 - **Server Machine 1** (ระบบที่มีช่องโหว่) นักเรียน 1 คน ในกลุ่ม ทำหน้าที่เปิดเครื่องตนเองเป็นแม่ข่ายรันไฟล์ `server_vulnerable.py` ซึ่งเป็นระบบที่ไม่ได้รับการปกป้องด้วย Lock
 - **Server Machine 2** (ระบบที่ปลอดภัย) นักเรียน 1 คน ในกลุ่ม ทำหน้าที่เปิดเครื่องตนเองเป็นแม่ข่ายรันไฟล์ `server.py` ซึ่งเป็นระบบที่ได้รับการแก้ไขปัญหาด้วย `asyncio.Lock`
 - **Client Hunters** (ลูกข่ายผู้แย่งชิง) นักเรียนทุกคนในกลุ่ม (รวมถึงผู้รับบท Server) จะต้องใช้เครื่องของตนเองรันไฟล์ `client.py` เพื่อยิง request แย่งชิงคูปอง (claim) ไปยัง Server เครื่องเพื่อนพร้อมกัน

กฎการกำหนดคลังคูปอง (Dynamic Inventory Constraint)

- เพื่อจำลองสถานะการแย่งชิงคูปองแบบต่อเนื่อง โดยอนุญาตให้นักเรียน 1 คนสามารถเก็บคูปองได้สูงสุดคนละ 2 ใบ ให้กำหนดจำนวนคูปองรวมใน Server ตามสูตร

$$\text{จำนวนคูปองรวมในคลัง} = (\text{จำนวนสมาชิกในกลุ่ม} \times 2) - 1$$

- ตัวอย่างผลลัพธ์ที่ถูกต้อง
 - กลุ่ม 5 คน: มีคูปอง 9 ใบ → นักเรียน 4 คนจะได้คนละ 2 ใบ และมี 1 คนได้เพียง 1 ใบ
 - กลุ่ม 6 คน: มีคูปอง 11 ใบ → นักเรียน 5 คนจะได้คนละ 2 ใบ และมี 1 คนได้เพียง 1 ใบ

การทดลองที่ 1: การสังเกตข้อผิดพลาดบน server_vulnerable.py

1. ให้ Server Machine 1 ตรวจสอบ IP Address ของเครื่องตนเองและประกาศให้เพื่อนในกลุ่มทราบ
2. ให้สมาชิกทุกคนปรับแต่ง SERVER_URL ในไฟล์ client.py ให้เรียกไปยัง IP ของ Server Machine 1
3. นัดเวลาทำการยิง Request POST /claim เข้าไปยัง Server พร้อมกันเพื่อเก็บคูปองสองใบ
4. สังเกตผลลัพธ์ที่เกิดขึ้น ทั้งใน Console ฝั่ง Server (เช่น การเกิด IndexError หรือ CRASH_BUG) และผลลัพธ์ฝั่ง Client (เช่น การได้รับคูปองเกินสิทธิ์ คูปองซ้ำรหัส หรือจำนวนคูปองในคลังติดลบ)

การทดลองที่ 2: การแก้ไขและยืนยันความถูกต้องบน server.py

1. เปลี่ยนไปเชื่อมต่อกับ Server Machine 2 ซึ่งมีการนำ `asyncio.Lock()` มาครอบครอบคลุมโซนการตรวจสอบและตัดจำนวนคุปอง (Critical Section)
2. ทำการยิง Request พร้อมกันอีกครั้งภายใต้เงื่อนไขเดิม (เก็บคุปองสูงสุดคนละ 2 ใบ)
3. บันทึกผลลัพธ์และยืนยันความถูกต้อง โดยระบบจะต้องแจกคุปองได้ครบตามจำนวน $(N \times 2) - 1$ ไม่มีการแจกคุปองซ้ำห้ส นักเรียนทุกคนต้องได้คุปองครบ 2 ใบ ยกเว้นซ้ำที่สุด 1 คนที่จะได้เพียง 1 ใบ และผู้ที่พยายามขอ ใบที่ 3 หรือผู้ที่มา claim ตอนคุปองหมดแล้วจะต้องได้รับสถานะปฏิเสธอย่างถูกต้อง 100%

เงื่อนไขการส่งงานรายบุคคล (Submission Deliverables)

- แม้นักเรียนจะทำการทดลองร่วมกันเป็นกลุ่ม แต่นักเรียนทุกคนจะต้องส่งไฟล์ซอร์สโค้ดฉบับสมบูรณ์เป็นรายบุคคล ประกอบด้วยไฟล์ดังต่อไปนี้
 1. **server_vulnerable.py** โค้ด Server ฝั่ง asynchronous ที่ไม่มีการใช้ Lock (แสดงจุดเสี่ยง Race Condition)
 2. **server.py** โค้ด Server ที่ได้รับการแก้ไขปัญหา Race Condition ด้วย `asyncio.Lock` เรียบร้อยแล้ว (ตั้งค่าคูปองแบบ $N \times 2 - 1$)
 3. **client.py** โค้ดฝั่ง Client ที่เขียนควบคุมการส่ง HTTP Request แบบ Concurrent ด้วย `httpx.AsyncClient` เพื่อพยายามเก็บคูปองให้ได้ 2 ใบ